2.1–2.2 · What learning replaces in the stack; frames, rotations, and the notation for the semester

Sessions: 2.1 (idea A: for each box of the stack, what the classical method assumes to know, and which learned component replaces, shortcuts, or keeps it), 2.2 (idea B: frames and rotations, written in the monogram notation we use until December, with one worked example). **Prerequisites established earlier:** the stack diagram and its five boxes (1.2), the symbol preview `q`, `φ`, `u`, `o_t`, `x_t` (Day 1 notes §3). **Notation used:** frames `W`, `T`, `M`, `G`, `O`; `^A p^B_C`, `^A R^B`, `^A X^B` as in `notation.md` §2.1; `q`, `φ(q)`. **New symbols introduced today** (added to `notation.md` §2.1 on 2026-09-04): the elementary rotations `R_x(θ)`, `R_y(θ)`, `R_z(θ)`; the skew matrix `[û]_×`; the unit quaternion `r = (w, x, y, z)`. The rotation angle is written `θ` here; `θ` is also the symbol for learned parameters in the learning block (Day 1 notes §3). Ruling R5 (accepted 2026-09-05): keep both, never on the same slide; `ϑ` if one document needs both.

0. What you will be able to do afterwards

- Take any box of the stack diagram and state (i) what the classical method in that box assumes to know, (ii) which learned component in this course replaces or shortcuts it and in which session, and (iii) what that learned component silently assumes instead.
- Read and write monogram notation without hesitation: given `^W p^T`, `^W R^T`, and `^T p^M`, compute `^W p^M`; given `^W p^M`, recover `^T p^M`; say what changes when only the subscript changes.
- Derive the two-dimensional rotation matrix from the angle-sum identities, state the three elementary three-dimensional rotations and the two defining properties of a rotation matrix, and write the same rotation as a rotation matrix, an axis and angle, a unit quaternion, and a RAI `.g` pose in both spellings.

1. Motivation and analogy

On Day 1 the classical manipulation stack was a drive to a new address: the map is the set of models the stack needs, the address is the goal, the route is the reference motion, the steering is the controller, and the eyes are perception. A learned policy supplies driving decisions from experience, while the rest of the system may still use a map. Today the analogy does two jobs.

Session 2.1 asks what the classical stack must know before it can act — what is printed on its map — and when that knowledge is wrong. A classical stack needs the geometry of the arm, the shapes of everything it may hit, and a current estimate of where everything is. Those three things are the map. With a rigid no-slip grasp, the mug moves with the hand, and the arm's model must account for the mug's mass and shape. A road map treats its connections as fixed, but contact can change the robot's motion constraints. This is where the analogy breaks. Predicting those changes makes model-based manipulation harder and motivates learned components. Session 2.1 identifies which assumptions the task needs and what happens when they fail.

Session 2.2 uses the other half of the analogy: directions relative to landmarks. "The mug is fifteen centimetres along the tray's long edge and five centimetres in" is a position relative to the tray, expressed in the tray's own directions. "The mug is at grid square (0.50, 0.32) on the table" is the same mug in the table's coordinates. Every box of the stack talks to its neighbours in one of these two ways, and converting between them is one rotation and one translation. The monogram notation exists so that nobody has to guess which landmark and which set of directions a number refers to.

Where the frames half of the analogy breaks. On a road map, north is north everywhere: one frame is enough. On a robot every link, the camera, and every object carries its own frame, and all of them move. Also, a landmark on a map has a position but no orientation; a mug has both, and a mug upside down is at the same position as a mug upright. This is why we must carry rotations along with positions from the very first day.

Part I · Session 2.1 — Which boxes learning replaces, shortcuts, or keeps

Source: deck 03 p.65–71 (Toussaint's closing slides of the motion-planning lecture) and deck 01 p.21. Those slides assumed the audience had already seen inverse kinematics, trajectory optimization, and controllers. In this course those arrive in weeks 3–4, so the material is re-pitched here at the level of the five boxes from Day 1 (decision of 2026-09-04). Nothing below needs an equation from later weeks.

I.0 Prerequisite ramp: the words this session uses before it can define them

Session 2.1 is a map of the whole course, and a map has to name places you have not been to yet. Every word below is named again, properly, in the session that owns it; the sentences here exist only so that nothing on today's slides is a blank. The slide

called “The words on the map, read off one mug” is this subsection compressed into seven lines. Read this once now and once after the lecture, and do not try to memorise any of it.

A **rigid body** is a solid part that does not bend, so that knowing where one point of it is and which way it is turned tells you where all of it is. A **multibody system** is a collection of rigid bodies — the arm’s links, the table, the mug — together with the **joints** that say which body is attached to which and how it is allowed to move. Each joint contributes one number, an angle for a hinge or a distance for a slider, and the whole list of those numbers is written `q`: a seven-joint arm has seven of them, and giving `q` fixes the shape of the entire arm in space. A **pose** is where a body is together with which way it is turned; a mug standing up and the same mug lying down are at the same position but not in the same pose, which is why a position by itself is never enough. A body floating free in space has six **degrees of freedom**: three numbers to say where it is and three to say which way it is turned. Its **mass** says how hard it is to get moving in a straight line, and its **inertia** says the same thing for turning, which depends not only on how heavy the body is but on how that weight is spread out — a long bar is harder to spin about its middle than a compact ball of the same mass. Inertia appears in this course only in the list of things a controller must be told; we never compute with it.

A **solver** is a program you hand a goal and a model, and which returns numbers that meet the goal. It cannot read the sentence “grasp the mug”, so the goal has to be rewritten in one of two forms. A **cost** is a single number saying how bad a candidate answer is, so that smaller is better and the solver’s job is to make it small: the squared distance between the gripper and the handle is a cost. A **constraint** is a condition the answer must satisfy exactly, such as “the fingers do not pass through the table”. **Differentiable** means that the formula for the cost also tells the solver how the number changes when each entry of `q` changes a little, which is what lets it improve an answer instead of guessing again; week 3 is about exactly this. Finally, a **state estimate** is the robot’s current best guess, computed from its sensors, of the numbers the solver needs: its own `q` (which the joint encoders measure very well), where each object is and which way it is turned (which a camera measures much less well), and parameters such as friction. The word *estimate* is doing real work: none of these numbers is known, all of them are inferred, and the second challenge of session 2.1 is what happens when one of them is wrong.

Word used on a slide today	What it means, in one line	Where the course teaches it properly
multibody system	the solid parts of the scene plus the joints between them	2.3, as a kinematic tree

Word used on a slide today	What it means, in one line	Where the course teaches it properly
joint angles `q`	one number per joint; the whole list fixes the arm's shape	2.3
pose	where a body is *and* which way it is turned	2.2, after the break today; as a 4×4 matrix in 2.3
degrees of freedom	how many numbers it takes to say where something is	2.3, and 4.4 as configuration space
mass, inertia	how hard to get moving; how hard to get turning	never computed here; 4.3 needs them
cost	one number saying how bad an answer is, to be made small	3.1
constraint	a condition the answer must meet exactly	4.1
differentiable	the formula also says which way is better	3.1
solver	a program that turns a goal plus a model into numbers	3.1–3.2
state estimate	the current guess, from sensors, of what the solver needs	5.2, 6.2
flange	the flat plate at the end of the arm that the hand bolts onto	nowhere: it is a name, not a topic
rank (slide 8)	how many independent directions the mug can be pushed in	2.4
torque	the twisting effort a motor makes at a joint	4.3
policy	a function from what the robot sees now to the next command	7.1
value function `V`, `Q`	a learned answer to "how good is it to be here?"	6.4, 11.1

Six abbreviations appear on today's slides or in the reading. **IK** is inverse kinematics (week 3), **RL** is reinforcement learning (weeks 10–11), **MDP** is a Markov decision process (6.4), **VLM** is a vision-language model (6.3), **VLA** is a vision-language-action model (weeks 8–9), and **SLAM** is simultaneous localisation and mapping, which is mentioned once in §I.4 and is not a topic of this course.

I.1 The five boxes and the map each one reads

Box (Day 1)	Question it answers	What it must know: its part of the map
Perception	where is everything?	a camera model; the shapes or appearance of the objects it looks for
Goal (inverse kinematics)	which joint angles put the gripper there?	the kinematic structure of the arm: which link is attached to which, link lengths, joint types and limits
Reference motion	which path gets there without collision?	the shapes of every body in the scene, for collision checking; the same kinematics
Controller	which torque now, so that we follow the path?	masses and inertias of the links, friction, motor limits
Motors	—	(mechanical design; out of scope)

Deck 03 p.65 states the same thing in three lines, which we adopt as the definition of "the map":

1. **The structure of the multibody system, all shapes, all inertias.** Which bodies exist, how they are jointed, how long each link is, what each body looks like from the outside (for collisions), how heavy it is and how its mass is distributed (for dynamics).
2. **Every goal or objective written as a differentiable cost or constraint.** "Grasp the mug" must become a statement like "the gripper frame must coincide with this pose" or "the distance between these two surfaces must be zero", so that a solver can work on it.
3. **A state estimate.** The current joint angles `q`, the poses of the objects, and, when dynamics matter, parameters such as friction, all estimated from sensors.

I.2 When the map is good enough

Deck 03 p.66 and p.70 make a point that is easy to skip and matters for the whole course: **if we only care about the robot itself, the three assumptions are fine.** The arm's structure is known from the manufacturer's data sheet, its shapes are known, its joint encoders measure `q` to a fraction of a degree. For moving an arm through free space to a pose, or along a path around known obstacles, reliable solvers exist: inverse kinematics (week 3), trajectory optimization and sampling-based planning (week 4), tracking controllers (4.3). Toussaint's words: "many aspects of robotics can really well be addressed with classical model-based solvers", and "it is sometimes non-sensical to explore learning methods for these aspects". The course's first learning outcome, building the classical stack, exists so that you can recognise these cases and not learn what a solver already does perfectly.

I.3 Challenge 1: interaction with the environment

Things change when the task is about the environment and not about the robot (deck 03 p.66–68). Two examples.

Locomotion. A walking robot generates its motion through contact forces with the ground. Its base is not bolted to the world, so its dynamics are those of a free-floating body plus legs. Contact forces switch on and off: a foot either pushes or it does not, and nothing in between. The dynamics therefore have a discrete, on–off structure layered on the continuous one. Solvers exist, but they are much more complicated than for a bolted-down arm. Deck 03's verdict: "more complicated than a vanilla robot, but still doable".

Manipulation. A loose mug has its own degrees of freedom and no joint motors. In our example it is supported and stationary before the grasp, so changing the arm's joint angles alone does not move it. A rigid no-slip grasp then fixes the mug's pose relative to the hand, so the mug follows the hand. The mug's sensitivity to the arm's joint angles is zero before this idealized grasp and nonzero after it. Session 2.4 calls that sensitivity its Jacobian. Gravity or an external push can move an ungrasped mug. Sliding or rolling contact also differs from a rigid attachment. More generally, object motion involves friction, stiction, slip, and contacts that are surfaces rather than points; the goal "grasp this shape correctly" is a far more complicated constraint than "gripper frame equals this pose"; and if the object is deformable, its shape itself becomes a set of degrees of freedom. One can still write down the physics of all of this and hand it to a solver, and people do. But the map is now redrawn at every contact event, and each redraw needs to be predicted correctly for the solver to work.

This is the analogy break of §1: **contact can change the motion constraints and force relationships that the solver must model.**

Computed, not asserted (`drake/00-contact-changes-the-map/contact_demo.py`, run on 2026-09-06 and re-run 2026-09-19 with Drake 1.46.0 on the Franka Panda model that Drake ships, with its hand; this demo's model, distinct from the official FR3 model used in the later FK walkthrough; see `drake/README.md`). The arm stands at its ready pose `q = (0, −0.785, 0, −2.356, 0, 1.571, 0.785)` rad with its flange at `^W p^G = (0.307, 0, 0.590)` m; a mug stands on the table between the open fingers, its centre 10.5 cm below the flange at `^W p^M = (0.307, 0, 0.485)` m (the Panda's palm ends about 5.8 cm below the flange and the fingers run from 5.8 cm below the flange to their tips at 11.2 cm, with the flat pads at 9.5–11.2 cm (measured on Drake's meshes, 2026-09-19), so an 8 cm tall mug this low has its top clear of the palm and its middle between the pads; until 2026-09-19 the demo put the mug 8 cm below the flange, inside the palm). Ask how much the mug's position changes per unit change of each joint angle: a 3 × 7 table of sensitivities, which session 2.4 will call the Jacobian `J_mug = ∂(^W p^M)/∂q`.

- **Before the grasp** the mug is fixed in the world: `J_mug` is the 3 × 7 zero matrix, rank 0. Turning the base joint by 0.2 rad moves the gripper by `(−0.006, 0.061, 0)` m and the mug by `(0, 0, 0)`.
- **The grasp** is modelled the way a classical solver models it: the mug is welded to the gripper at its current relative pose, `^G p^M = (0, 0, 0.105)` m along the flange axis. Nothing else in the scene changes.
- **After the grasp** `J_mug` is

  ```
  [  0       0.152   0      0.130   0      0.212   0 ]
  [  0.307   0       0.325  0       0.212  0       0 ]
  [  0      −0.307   0      0.472   0      0.088   0 ]
  ```

  with rank 3. The same 0.2 rad on the base joint now moves the mug by `(−0.006, 0.061, 0)` m, together with the gripper.

Same `q`, same joint change, same mug: only the map changed, from "the mug is part of the world" to "the mug is part of the arm". Two remarks. The first-order prediction `J_mug Δq = (0, 0.061, 0)` misses the actual motion by 6 mm because 0.2 rad is not a small step; a Jacobian is a first-order object, which 2.4 makes precise. And the weld is the idealisation: the friction, the slip, and the moment of switching between the two maps are what the solver has to be told about and cannot discover on its own. The script's `--meshcat` mode puts seven joint sliders and a "Grasp / release" button in the browser for the live version of this slide.

I.4 Challenge 2: state estimation

Everything in §I.1 needs a state estimate (deck 03 p.69): the robot's `q` (and, for a mobile robot, its pose), the poses of the objects, and the shapes, inertias, and friction of things the robot did not come with. Very good estimators exist, and many of them are optimization problems of the same kind as motion optimization: find the estimate most consistent with the sensor readings by minimising the mismatch between what the sensors read and what the model predicts they would read. Graph-based simultaneous localisation and mapping is the textbook example, and it will look familiar after week 3.

But estimating the pose of an unknown object from a camera, in clutter, with occlusions, is not a solved problem in the way inverse kinematics is. Deck 03 p.69 puts it as strongly as anything in the course: **"for general robotics, the need for state estimation is the main downside of model-based robotics solvers, and a core motivation for learning methods."** Keep this sentence; the perception weeks (5–6) and the imitation weeks (7–8) are both answers to it.

I.5 Which problems are easy and which are hard

Deck 03 p.70, as a rule of thumb for the rest of the course:

- **Easy** (in the sense that reliable model-based methods exist): everything about the robot itself, given the map. Kinematics, motion planning in a known scene, tracking a reference.
- **Hard** (the two challenges): predicting and controlling contact, and estimating the state of a scene the robot did not design.
- Robot learning and model-based solvers share their foundations: an optimal-control problem and a Markov decision process both formalise sequential action choice for an objective. Their assumptions about dynamics, observations and how solutions are obtained must be stated when comparing them (session 6.4).

So the question is never "learning or models" but "which component, and why".

I.6 What learning supplies, and what remains in the stack

Use the mug task to distinguish five roles for a learned component. A **model** predicts what happens, a **score** compares choices, a **proposal** supplies a candidate, a perception map estimates the scene, and a **policy** maps observations to actions. These roles differ in what the remaining software must do.

Learned output	Mug example	What can remain classical
Model or correction	Predict slip under a candidate motion	Planning with that prediction

Learned output	Mug example	What can remain classical
Score	Estimate which candidate grasp succeeds	Search or selection over candidates
Proposal or initial guess	Suggest a reaching posture	IK refinement and feasibility checks
Perception estimate	Infer the mug pose from an image	Planning and tracking control
Action	Choose the next motion from observations	The action interface and low-level controller

A value function estimates future return; an action-value function estimates it after a specified action. Either can help action selection, but a learned action-value function does not automatically remove the action-search problem. Neither is, in general, an inverse-dynamics map from a desired acceleration to a torque. These distinctions replace the inherited shorthand “learn Q instead of V” on the inverse-dynamics row. The value-method derivations belong to the later sequential-decision-making block, not this vocabulary introduction.

A proposal need not satisfy constraints without checking. An image-to-action policy can avoid an explicit object-pose estimate, but still has to handle incomplete and uncertain observations. An explicit model and a learned component both require validation. What we check depends on the component’s role.

Learned component	What we must validate
Dynamics model	Its predictions hold for the states and motions the deployed robot encounters.
Reward-based score or policy	The task score, called a reward, expresses the intended objective. Observations contain the information needed for its choices, and training supports those choices.
IK proposal	Its candidate goals and robot geometry are covered, and the retained stack checks constraints.
Perception map	Pose accuracy holds for the cameras, viewpoints and objects used at deployment.

The reward row applies to methods trained to improve that score. It does not require an explicit reward for supervised pose estimation or imitation from demonstrations. A new camera or object need not match the training examples exactly, but performance

under that change needs evaluation. Model adequacy, training coverage and interface conventions all matter.

I.7 Three questions to ask of every learned component

From now on, whenever a learned component appears in this course, we ask the same three questions and write the answers on one slide:

1. **What does it replace?** Identify the calculation the learned output supplies.
2. **What does it keep?** Identify the remaining software and its required inputs.
3. **What does it silently assume?** Identify the model and data conditions under which the complete system is expected to work.

Apply the questions to one estimator. A network predicts the mug's world-frame pose from an image. That pose goes to the existing inverse-kinematics solver, planner and controller. The camera then moves to a new viewpoint. Answer the three questions before reading on.

The network replaces the calculation that estimates the mug's pose from an image. The pose interface, inverse-kinematics solver, planner, controller and physical robot remain. They still require the estimated pose in the agreed world frame and units. Moving the camera changes the image-to-world relationship and can change the mug's appearance. Check pose error on representative images from the new viewpoint, then check grasp success through the retained stack. Training at one viewpoint establishes neither failure nor success at another. The estimation example in §I.4 explains why this matters: a solver can accurately solve the problem defined by an incorrect pose estimate.

Session 5.1 applies this to learned samplers for planning, 6.2 to learned perception, 7.1 to imitation learning, 8.3 to VLA anatomy, and 10.4 to reinforcement learning as post-training.

Part II · Session 2.2 — Frames, rotations, and monogram notation

Source: deck 02 p.6–18. What is kept: the "step 1: kinematic frames" framing (p.6), the monogram anatomy (p.10), the two in-class questions (p.11–12, replaced by one that can be checked from a figure), the two-dimensional derivation (p.13), the elementary rotations (p.14), and the three side-information slides on Euler angles, rotation vectors, and quaternions (p.15–17), compressed. What is new: the worked example with a mug on a tray (which differs deliberately from assignment A1's ball-

on-a-tray scene), the reading exercise, the explicit "same arrow, different numbers" point, the RAI `.g` spellings, and the interactive module `modules/frames-monogram/`.

2. Prerequisite ramp: a vector and its coordinates are different things

Everything today rests on one distinction. A **vector** is an arrow: it has a length and a direction and exists without any numbers. Its **coordinates** are three numbers that depend on which three reference directions we measure it against. Change the reference directions and the numbers change; the arrow does not.

Take three unit vectors `e_1, e_2, e_3` that are mutually perpendicular (an **orthonormal basis**): `e_i · e_j = 1` if `i = j` and `0` otherwise. Any vector `p` can be written as

```
p = p_1 e_1 + p_2 e_2 + p_3 e_3,        p_i = e_i · p,
(1)
```

and the triple `(p_1, p_2, p_3)` are its coordinates in that basis. The second half of (1) follows from the first by taking the dot product with `e_i` and using orthonormality; it says that a coordinate is a projection. A **matrix times a vector** is a linear combination of the matrix's columns with the vector's entries as weights, which is the one fact about matrices used in §4:

```
M v = v_1 (column 1 of M) + v_2 (column 2 of M) + v_3 (column 3 of
M).             (2)
```

Small example. In the plane, let `e_1 = (1, 0)`, `e_2 = (0, 1)`, and let a second basis be `e'_1 = (0, 1)`, `e'_2 = (−1, 0)`, which is the first basis turned by 90°. The arrow that has coordinates `(1, 0)` in the first basis is `e_1`; its coordinates in the second basis are `(e'_1 · e_1, e'_2 · e_1) = (0, −1)`. Same arrow, different numbers. That is the whole of today in one line.

3. Formal setup

3.1 Frames

A **frame** is an origin (a point) together with a right-handed orthonormal basis attached to it. "Right-handed" means `e_1 × e_2 = e_3`: if the first axis points along your right thumb and the second along your index finger, the third points along your middle finger. Frames are named by single capital letters. This semester the recurring ones are `W` (the world, fixed to the table or the room), `G` (the gripper), `O` (an object), `C` (the camera in the perception weeks), and, today, `M` (the mug) and `T`

(the tray). Every rigid body in a scene carries a frame, and RAI's `.g` files are exactly lists of frames with their shapes and their poses relative to a parent frame (deck 02 p.26; session 2.3).

3.2 Monogram notation (Drake's convention; notation ruling R3)

Three quantities carry the course. Their names encode three facts each (deck 02 p.10).

```
^A p^B_C     the position of B relative to A, expressed in the axes
of C
^A R^B       the rotation of frame B relative to frame A
^A X^B       the pose of B relative to A: the pair (^A R^B, ^A p^B)
```

Reading rules:

- The **left superscript** is the reference: where the arrow starts (for `p`) or the frame we measure against (for `R`, `X`). The **right superscript** is the target: where the arrow ends, or the frame being described.
- The **subscript** on `p` says **in whose axes the three numbers are written**. `^A p^B_A` and `^A p^B_C` are the same arrow with different numbers. When the subscript is omitted it equals the left superscript: `^A p^B` means `^A p^B_A`.
- `^A R^B` needs no subscript. Its columns are the axes of `B` written in the coordinates of `A` (proved in §4.2), so it already says in which frame it lives.
- `^A X^B` is introduced today only as a name for the pair; the 4×4 matrix and the rule `^A X^C = ^A X^B · ^B X^C` are session 2.3.

In Drake code, and in the course's Python, the same objects are written `p_AB_C`, `R_AB`, `X_AB`, reading the superscripts left to right. The `frames_example.py` file in §6 uses these names, so that what you read on a slide is what you type.

3.3 Rotation matrices

A **rotation matrix** is a real 3×3 matrix `R` with

```
Rᵀ R = I    and    det R = +1.
(3)
```

The set of all such matrices is written `SO(3)` (the special orthogonal group; "special" stands for `det = +1`). The first condition says the columns are orthonormal; the second excludes reflections, which also have orthonormal columns but have `det = −1` and turn a right-handed frame into a left-handed one. Both conditions are derived, not decreed, in §4.2. The **elementary rotations** about the three axes by an angle `θ` are

```
R_z(θ) = [ cos θ  −sin θ  0 ]      R_x(θ) = [ 1   0       0    ]
R_y(θ) = [  cos θ  0  sin θ ]
         [ sin θ   cos θ  0 ]               [ 0  cos θ  −sin θ ]
[   0     1   0    ]
         [   0       0    1 ]               [ 0  sin θ   cos θ ]
[ −sin θ  0  cos θ ]     (4)
```

and they are derived in §4.3. Angles are in radians in every formula and in all code; degrees appear only in prose and in RAI's `d(...)` spelling.

3.4 Three other ways to write a rotation (side information; deck 02 p.15–17)

A rotation matrix has nine numbers for three degrees of freedom; the six constraints in (3) remove the redundancy. Three smaller representations are in daily use, and you will meet all of them in the simulators.

Euler angles. Three successive rotations about coordinate axes, for example yaw about `z`, then pitch about the new `y`, then roll about the new `x`: `R = R_z(ψ) R_y(ϑ_p) R_x(ϕ)`. Compact and readable, but the parametrization has a singularity: when the pitch is `±90°`, the yaw and roll axes align, so their local effects are dependent. The Euler-angle coordinates lose an independent direction of change, while the body retains all three rotational freedoms. This is **gimbal lock**; §5 (g) shows it numerically. Rotation matrices and unit quaternions avoid this coordinate singularity. (The symbols `ψ`, `ϑ_p`, `ϕ` are used only in this paragraph.)

Rotation vector, or axis and angle. Any rotation is a rotation by some angle `θ` about some unit axis `û` (Euler's rotation theorem, not proved here). The **rotation vector** `θ û` holds both: its length is the angle in radians and its direction is the axis. (It is often written `ω` in textbooks; this course keeps `ω` for angular velocity, so the rotation vector is written out as `θ û`.) The matrix is recovered by **Rodrigues' formula**, derived in §4.6:

```
R(û, θ) = I + sin θ [û]_× + (1 − cos θ) [û]_×²,    [û]_× = [  0    −u_z   u_y ]
                                                           [  u_z   0    −u_x ]
                                                           [ −u_y   u_x   0   ]
          (5)
```

where `[û]_×` is the skew-symmetric matrix with `[û]_× v = û × v`. For the identity rotation, the separate unit axis is undefined, but the rotation vector is zero and its map to a rotation matrix is smooth there. This local smoothness does not make the encoding globally unique.

Unit quaternion. Four numbers `r = (w, x, y, z)` with `w² + x² + y² + z² = 1`, built from the axis and angle as

```
r = ( cos(θ/2),  sin(θ/2) û ),
(6)
```

so `w = cos(θ/2)` and `(x, y, z) = sin(θ/2) û`. No gimbal lock, four numbers instead of nine, composition by a cheap product, and the matrix is

```
R(r) = [ 1 − 2(y² + z²)     2(xy − wz)       2(xz + wy)   ]
       [   2(xy + wz)     1 − 2(x² + z²)     2(yz − wx)   ]
       [   2(xz − wy)       2(yz + wx)     1 − 2(x² + y²) ]
(7)
```

Formula (7) is stated, not derived, in this course; §6 verifies it numerically on the worked example, and Lynch and Park's *Modern Robotics*, Appendix B, has the derivation. Two quaternions `r` and `−r` give the same rotation; keep that in mind when comparing them.

What RAI stores. Checked against the `robotic` tutorial `config_1_intro` and the model file `rai-robotModels/scenarios/pandaSingle.g` on 2026-09-04: a frame's pose is the 7-vector `[p_x p_y p_z q_w q_x q_y q_z]`, position first, then the quaternion **with `w` first**; the tutorial states "we use the (w, x, y, z) convention", and any quaternion you set is normalised internally. In a `.g` file a pose can also be written as text, `"t(x y z) d(angle_in_degrees a_x a_y a_z)"`: translate, then rotate by the angle about the axis. Drake uses `w` first as well. SciPy's `Rotation.as_quat()` and ROS messages put `w` **last**, `(x, y, z, w)`, which is the single most common source of silently wrong orientations when two libraries meet.

4. Derivation

4.1 The two-dimensional rotation matrix from the angle-sum identities (deck 02 p.13)

Take a point `p` at distance `L` from the origin, at angle `γ` from the first axis, so that

```
p = (p_x, p_y) = (L cos γ, L sin γ).
(8)
```

Rotate it counter-clockwise by `θ` about the origin. It stays at distance `L` and its angle becomes `γ + θ`:

```
p' = (p'_x, p'_y) = (L cos(γ + θ), L sin(γ + θ)).
(9)
```

Expand with the angle-sum identities `cos(γ + θ) = cos γ cos θ − sin γ sin θ` and `sin(γ + θ) = sin γ cos θ + cos γ sin θ`, then substitute (8) to eliminate `L` and `γ`:

```
p'_x = L cos γ cos θ − L sin γ sin θ = p_x cos θ − p_y sin θ,
p'_y = L sin γ cos θ + L cos γ sin θ = p_x sin θ + p_y cos θ.
(10)
```

Both lines are linear in `(p_x, p_y)`, so they are one matrix product:

```
[ p'_x ]   [ cos θ  −sin θ ] [ p_x ]
[ p'_y ] = [ sin θ   cos θ ] [ p_y ],        R(θ) = [ cos θ  −sin θ ]
                                                    [ sin θ   cos θ
].                (11)
```

Figure `fig/rot2d.png` (from `code/make_figures.py`) draws (8) and (9) for `γ = 20°`, `θ = 35°`, `L = 1 m`.

4.2 What the columns mean, and the two properties in (3)

Apply (11) to the basis vectors. `R(θ) (1, 0) = (cos θ, sin θ)` is the first column; `R(θ) (0, 1) = (−sin θ, cos θ)` is the second. So **the columns of a rotation matrix are the rotated basis vectors**, which, read as a change of frame, means: the columns of `^A R^B` are the axes of frame `B` written in the coordinates of frame `A`. That is why `^A R^B` needs no "expressed in" subscript.

The two columns have unit length (`cos² θ + sin² θ = 1`) and are perpendicular (`−cos θ sin θ + sin θ cos θ = 0`), which is the statement `Rᵀ R = I`, and the determinant is `cos² θ + sin² θ = 1`. Both properties in (3) are therefore consequences of "a rotation preserves lengths and angles and does not flip the plane", not extra assumptions.

`Rᵀ R = I` has an immediate use: **the inverse of a rotation is its transpose**, `R⁻¹ = Rᵀ`, and in two dimensions `R(θ)ᵀ = R(−θ)`, which you can read off (11). No matrix inversion is ever needed to undo a rotation.

Active and passive readings. Equation (11) answers two different questions with the same matrix. Active: "where does the point `p` go when I rotate it by `θ`?" (the point moves, the frame stays). Passive: "a point has coordinates `^B p` in a frame `B` that is rotated by `θ` relative to `A`; what are its coordinates in `A`?" Answer: `^A p = ^A R^B ^B p`, with `^A R^B = R(θ)` (the point stays, the frame changes). In robotics we almost always ask the passive question. Converting back from A coordinates to B coordinates uses R transpose; state the input and output frames explicitly. It is worth saying aloud which question you are asking; §7 lists the confusion as the most common error of the week.

4.3 The three elementary rotations in three dimensions (deck 02 p.14)

A rotation about the `z` axis leaves `z` coordinates unchanged and rotates the `(x, y)` plane as in (11); embedding (11) in the upper-left block gives `R_z(θ)` in (4). Rotations about `x` and `y` follow by renaming the axes cyclically (`x → y → z → x`): for `R_x(θ)` the `(y, z)` plane rotates and `x` is fixed; for `R_y(θ)` the `(z, x)` plane rotates and `y` is fixed, which is why the sign pattern of `R_y` looks different: with the cyclic order the "first" axis of that plane is `z`, not `x`. Deck 02 p.14 suggests the right check: set `θ = 90°` and look at the columns. `R_z(90°)` has columns `(0, 1, 0)`, `(−1, 0, 0)`, `(0, 0, 1)`: the old `x` axis now points along `y`, the old `y` axis along `−x`, `z` is unchanged. If the columns of a matrix you wrote do not tell that story, the matrix is wrong.

4.4 Changing the reference frame: rotation and translation together

Let frame `B` sit at position `^A p^B` relative to `A` with orientation `^A R^B`, and let a point `P` have known coordinates `^B p^P` in `B`. The arrow from `A`'s origin to `P` is the arrow from `A` to `B`'s origin followed by the arrow from `B`'s origin to `P` (vectors add head to tail). The first arrow is already in `A`'s coordinates. The second is known in `B`'s coordinates, and §4.2 converts it: `^B p^P_A = ^A R^B ^B p^P`. Hence

```
^A p^P = ^A p^B + ^A R^B ^B p^P.
(12)
```

This is the one equation of session 2.2. Its inverse, using $R^{-1} = R^{T}$, is

```
^B p^P = (^A R^B)ᵀ ( ^A p^P − ^A p^B ),
(13)
```

subtract the offset first, then rotate back. Read (12) once more with the subscripts made explicit: `^A p^P_A = ^A p^B_A + ^B p^P_A`, and `^B p^P_A = ^A R^B ^B p^P_B`. The middle term is the "same arrow, different numbers" case: the arrow from the tray to the mug, first in the tray's axes, then in the world's.

4.5 Composing rotations

If `C` is rotated by `^B R^C` relative to `B`, and `B` by `^A R^B` relative to `A`, then a vector with coordinates `^C v` has coordinates `^B R^C ^C v` in `B` and therefore `^A R^B (^B R^C ^C v)` in `A`. So

```
^A R^C = ^A R^B · ^B R^C,
(14)
```

and the frame letters must match in the middle, the way units cancel. Matrix products do not commute, so `^A R^B ^B R^C ≠ ^B R^C ^A R^B` in general; the second product is not even a meaningful object because the inner letters do not match. Example with numbers: `R_z(90°) R_x(90°)` has first column `(0, 1, 0)`, while `R_x(90°) R_z(90°)` has first column `(0, 0, 1)`: rotating about `x` then `z` is not the same as `z` then `x`. Rotations about a single fixed axis do commute (`R_z(a) R_z(b) = R_z(a + b)`), which is why the worked example in §5 (d) composes to the identity, and why the two-dimensional module in §6 cannot show the order effect; the three-dimensional module `rotations-3d` (§6, tab 2) shows it with two books, and the Drake notebook planned for 2.4 does as well.

4.6 Rodrigues' formula (5)

Let `û` be a unit axis and `v` any vector. Split `v` into the part along the axis and the part perpendicular to it:

```
v_∥ = (û · v) û = (û û^T) v,        v_⊥ = v − v_∥.
(15)
```

A rotation about `û` leaves `v_∥` unchanged and turns `v_⊥` in the plane perpendicular to `û`. In that plane, `v_⊥` and `û × v` are perpendicular and have the same length (because `|û × v| = |v| sin ∠(û, v) = |v_⊥|` and `û × v = û × v_⊥`), so they play the roles of the two axes in (11):

```
R v_⊥ = cos θ v_⊥ + sin θ (û × v).
(16)
```

Adding the unchanged parallel part and writing the cross product as `[û]_× v`:

```
R v = (û û^T) v + cos θ (I − û û^T) v + sin θ [û]_× v.
(17)
```

Finally, the identity `[û]_×² = û û^T − I` (check any component: for instance the (1,1) entry of `[û]_×²` is `−u_y² − u_z² = u_x² − 1`, using `|û| = 1`) turns `û û^T` into `I + [û]_×²`, and collecting terms gives

```
R = I + sin θ [û]_× + (1 − cos θ) [û]_×²,
(5)
```

as claimed. Check: `û = (0, 0, 1)` gives `[û]_×` with entries `−1` at (1,2) and `+1` at (2,1), `[û]_×² = diag(−1, −1, 0)`, and (5) becomes `R_z(θ)`; §6 confirms this to machine precision for `θ = 30°`.

4.7 Gimbal lock, in one paragraph

For yaw–pitch–roll `R = R_z(ψ) R_y(ϑ_p) R_x(ϕ)` at pitch `ϑ_p = 90°`, the matrix `R_y(90°)` maps the body `x` axis onto the world `−z` axis (its first column is `(0, 0, −1)`). Positive roll turns about world negative z, while positive yaw turns about world positive z. Their effects oppose each other, so only yaw minus roll matters at this pitch. The Euler-angle map loses an independent direction of change; the body retains three rotational freedoms. Section 5 (g) checks this numerically: the matrices for `(ψ, ϕ) = (10°, 0°)` and `(20°, 10°)` agree to `10⁻¹⁷`.

5. Worked example: a mug on a tray on a table

The scene (`code/tray_scene.g`, figure `fig/tray_topview.png`): the world frame `W` is at a corner of the table, `z` up. The tray frame `T` is at the centre of the tray, which sits at

```
^W p^T = (0.40, 0.20, 0.75) m,    ^W R^T = R_z(30°).
```

The mug frame `M` is at the centre of the mug, which stands on the tray at

```
^T p^M = (0.15, 0.05, 0.05) m,    ^T R^M = R_z(−30°)
```

(its handle is turned back by 30°, so that it ends up aligned with the table). Every number below is printed by `code/frames_example.py`, rounded to four decimals.

(a) Where is the mug in world coordinates? By (12), with `cos 30° = 0.8660` and `sin 30° = 0.5000`:

```
^W R^T ^T p^M = [ 0.8660  −0.5000  0 ] [ 0.15 ]   [ 0.8660·0.15 −
0.5000·0.05 ]   [ 0.1299 − 0.0250 ]   [ 0.1049 ]
                [ 0.5000   0.8660  0 ] [ 0.05 ] = [ 0.5000·0.15 +
0.8660·0.05 ] = [ 0.0750 + 0.0433 ] = [ 0.1183 ]
                [   0        0     1 ] [ 0.05 ]   [            0.05
]   [      0.05       ]   [ 0.05   ]

^W p^M = ^W p^T + ^W R^T ^T p^M = (0.40 + 0.1049, 0.20 + 0.1183, 0.75
+ 0.05) = (0.5049, 0.3183, 0.80).
```

(b) Same arrow, different numbers. The arrow from the tray's origin to the mug is `^T p^M_T = (0.15, 0.05, 0.05)` in the tray's axes and `^T p^M_W = (0.1049, 0.1183, 0.05)` in the world's axes. It is one arrow; only the subscript changed. Its length is the same in both: `√(0.15² + 0.05² + 0.05²) = √(0.1049² + 0.1183² + 0.05²) = 0.1658 m`.

(c) The inverse problem. A camera reports the mug at `^W p^M = (0.5049, 0.3183, 0.80)` and the tray at the pose above. Where is the mug on the tray? By (13):

```
^W p^M − ^W p^T = (0.1049, 0.1183, 0.05),
(^W R^T)ᵀ (0.1049, 0.1183, 0.05) = ( 0.8660·0.1049 + 0.5000·0.1183,
−0.5000·0.1049 + 0.8660·0.1183,  0.05 )
                                 = ( 0.0908 + 0.0592,  −0.0525 +
0.1025,  0.05 ) = (0.1500, 0.0500, 0.05).  ✓
```

(d) Composition. `^W R^M = ^W R^T ^T R^M = R_z(30°) R_z(−30°) = R_z(0°) = I`. The mug's axes are aligned with the table's, as intended.

(e) Properties. `(^W R^T)ᵀ ^W R^T = I` and `det ^W R^T = 1.000000`, printed by the code.

(f) The same rotation three more ways. Axis `û = (0, 0, 1)`, angle `30°`: Rodrigues' formula reproduces `R_z(30°)` exactly. Quaternion by (6): `r = (cos 15°, 0, 0, sin 15°) = (0.9659, 0, 0, 0.2588)`, unit norm, and (7) reproduces `R_z(30°)` exactly. In RAI the tray's pose is therefore either

```
X: "t(0.40 0.20 0.75) d(30 0 0 1)"       or       X: [0.40 0.20 0.75
0.9659 0 0 0.2588]
```

and the mug's line in the same file is

```
mug (tray): { Q: "t(0.15 0.05 0.05) d(−30 0 0 1)" }
```

Each line of a `.g` file names a frame and, in parentheses, the frame it is placed in; `X:` gives the pose in the world and `Q:` the pose relative to that parent, so the mug's `Q:` is `^T X^M` with the position of (a) and the turn `R_z(−30°)` of the scene setup. The commands inside the quotes apply left to right, each in the frame the previous ones produced, which is why `t(...)` comes first. Loading the file and asking for the mug's world position, `C.getFrame('mug').getPosition()`, should return `^W p^M = (0.5049, 0.3183, 0.80)` of (a); this has not been run yet (§6).

(g) Gimbal lock. `R_z(10°) R_y(90°) R_x(0°)` and `R_z(20°) R_y(90°) R_x(10°)` differ by at most `1.1 × 10⁻¹⁷`: at pitch 90°, only yaw minus roll matters.

Reading exercise (slide 20, figure `fig/quiz_frames.png`). Which of the three is `^T p^M`, the mug relative to the tray, expressed in the tray's axes? (a) `(0.10, 0.12)`, (b) `(0.15, 0.05)`, (c) `(0.50, 0.32)`. Answer: **(b)**. Option (a) is the same arrow in the world's axes, `^T p^M_W`, rounded; option (c) is `^W p^M`, the mug relative to the world. Every wrong option is a real quantity with the wrong letters, which is how frame errors look in practice.

6. Code

Three files in `code/`, all run with the course environment (`SETUP.md`); the first two ran on 2026-09-04 on macOS with the numbers above.

- **`frames_example.py`** — NumPy implementation of §5 (a)–(g) with the elementary rotations, Rodrigues' formula, and the quaternion conversion as small functions; prints every number in §5. If `pydrake` is installed it repeats (a), (c), (d) with Drake's `RigidTransform`, using Drake's variable names `X_WT`, `p_TM`, so that the monogram notation on the slide and in the code coincide. Runtime: well under a second.
- **`make_figures.py`** — draws `fig/rot2d.png`, `fig/tray_topview.png`, and `fig/quiz_frames.png` with Matplotlib from the same numbers. Runtime: about two seconds.
- **`tray_scene.g`** — the scene as a RAI configuration file, with both pose spellings for the tray and a relative pose `Q:` for the mug. Syntax checked against `rai-robotModels/scenarios/pandaSingle.g` on 2026-09-04. **Not yet loaded on a Linux machine;** before R 3 it should be opened there with `C = ry.Config(); C.addFile('tray_scene.g')` and `C.getFrame('mug').getPosition()` compared with §5 (a). Not yet arranged with the TAs (`CHECKLIST.md` C3).
- **`drake/00b-when-the-map-is-wrong/map_error_demo.py`** (session 2.1, slide 9; added 2026-09-19) — the same Panda with Drake's hand model. The stack plans inverse kinematics on the *estimated* mug position (the true one plus an error Δ) and executes the plan exactly. What happens next is played out by three kinematic rules (no physics engine): the hand comes straight down onto the plan and **stops at the first contact** of a finger or the palm with the real mug (a Franka's collision reflex; a robot without one would push the mug over); if the mug is between the open fingers they close on it and, being mechanically coupled, centre it (a grasp); if the mug is out of reach they close on air. Print mode tabulates eight errors; the Meshcat mode has three sliders for Δ in centimetres, draws the belief as an orange ghost and the truth in teal, animates the approach and the closing, and colours the ghost green or red. Numbers (2026-09-19): the hand opens to 80 mm and the mug is 70 mm across, so the mug is between the open fingers only if the estimate is right to **5 mm along the closing axis** — 4 mm grasps, 6 mm puts a finger on the rim; the default 2 cm error stops the arm with a finger on the rim 4.6 cm above the planned grasp; along the finger pads about 4 cm is tolerated; a mug 2 cm *higher* than believed meets the palm. The hand is modelled as boxes (palm, two fingers with 2 cm wide pads), an approximation of the Franka hand; the closing-axis clearance is exact.
- **`drake/00-contact-changes-the-map/contact_demo.py`** (session 2.1, slide 8) — the Franka Panda with its gripper (hand model since 2026-09-19) and a mug that is welded to the world or to the gripper; prints the mug's 3 × 7 sensitivity table and its rank before and after the grasp (§I.3), and with `--meshcat` gives seven joint sliders and a "Grasp / release" button in the browser: the fingers close

on the mug and it follows the hand; release opens the fingers and the mug drops onto the table under the point where it was let go (a rule, not physics — the demo has no dynamics). Runs in the course `.venv` (`SETUP.md`); verified 2026-09-06 in print mode on the Panda model, and the interactive grasp logic is self-tested at the end of print mode since 2026-09-19.

Live demo: `modules/rotations-3d/index.html` (added 2026-09-19; slides 23, 28, 29). One file, three tabs, three.js from a CDN (needs a network connection the first time). Tab 1: an elementary rotation with its columns drawn as the body's axes, active versus passive as a switch, "set $\theta = 90°$" and a $\det = -1$ mirror. Tab 2: two books under the same two rotations in opposite orders (the order effect §4.5 says the 2-D module cannot show), and yaw–pitch–roll with drawn gimbal rings opening at the lock of §4.7. Tab 3: Rodrigues, axis–angle, the quaternion and both RAI spellings live, plus the `(w, x, y, z)` read as `(x, y, z, w)` ghost: the tray's quaternion becomes 150° about x. Every default number is printed by `modules/rotations-3d/check_numbers.py` and matches §4–§5. Companion notes: `modules/rotations-3d/notes.md`.

Live demo: `modules/frames-monogram/index.html`. A two-dimensional, single-file module: drag the tray frame and the mug, turn the tray with a slider, and read `^W p^T`, `^W R^T`, `^T p^M_T`, `^T p^M_W`, and `^W p^M` live, with the arrow from tray to mug decomposed along either frame's axes. Its default state is exactly the worked example, so the numbers on the screen are the numbers in §5 (a)–(c). Companion notes: `modules/frames-monogram/notes.md`.

7. Common failure modes

- **Reading the superscripts backwards.** `^T p^M` is the mug relative to the tray, never the tray relative to the mug. The arrow goes from the left letter to the right letter. Fix: say the sentence out loud, "position of M relative to T".
- **Dropping the subscript.** `(0.15, 0.05)` and `(0.1049, 0.1183)` are the same arrow; a program that adds `^T p^M_T` to `^W p^T` (without rotating first) puts the mug about 8.2 cm off: `(0.55, 0.25)` against `(0.5049, 0.3183)`, and reading `(0.1049, 0.1183)` as `^T p^M` is off by the same 8.2 cm. This is the reading exercise's option (a), and it is the most common bug in A1.
- **Composing in the wrong order or with unmatched letters.** In `^A R^B ^B R^C` the inner `B`s must match. If they do not, the product is not a rotation between the frames you think; check by reading the letters, not by looking at the numbers.
- **Confusing active and passive.** Same matrix, different question. Decide which question you are asking before you write `R p` or `Rᵀ p`. A quick test: apply the matrix to a point on one of `B`'s axes and see whether the answer lands where `B`'s axis is drawn.

- **Degrees in code.** `np.cos(30)` is the cosine of 30 radians. Every function in `frames_example.py` converts explicitly; RAI's `d(30 0 0 1)` is the only place where a degree appears raw.
- **Quaternion order.** RAI and Drake: `(w, x, y, z)`. SciPy and ROS: `(x, y, z, w)`. A quaternion read from one and written into the other rotates the object by a wrong angle about a wrong axis and raises no error.
- **A sign slip that makes a reflection.** If you build `R` by hand and `det R = −1`, one column has the wrong sign and your frame is left-handed. `det R` is a one-line check.

8. Where this sits in the stack

Frames are the language in which the boxes talk to each other. Perception outputs object poses `^W X^O`; the goal box needs a desired gripper pose `^W X^G` and the arm's forward kinematics `φ(q) = ^W X^G(q)` (session 2.3); the reference-motion box checks collisions between shapes whose poses are all `^W X^{(·)}`. A learned policy can omit an explicit object-pose estimate between its image input and action output. Its data and action interface still carry geometric conventions. An end-effector command needs a reference frame and units; a joint command needs the robot's joint ordering and units. A pose increment also needs a stated update rule. Session 8.3 asks which interfaces a particular model retains and how its actions reach the robot. The output format alone does not establish what geometry the network has learned internally or which deployment conditions its training covered.

9. Further reading (optional, never load-bearing)

- Russ Tedrake, *Robotic Manipulation* (online textbook), chapter 3, section "Monogram notation", and the Drake documentation page "Multibody notation": the source of the convention adopted in ruling R3.
- Kevin Lynch and Frank Park, *Modern Robotics*, chapter 3 (rotations, exponential coordinates) and Appendix B (quaternions): the derivations skipped in §3.4.
- Marc Toussaint, lecture notes *Robotics* (TU Berlin), chapter "Kinematics": the same material in RAI's language.
- Deck 02 p.6–18 (frozen record) for last year's slides on the same sessions.
- Deck 03 p.65–71 for the original wording of session 2.1's material.

Open questions and weak points

- **Session 2.1's one computation is the contact demo** (§I.3, slide 8). It models the grasp as a weld, which is the classical idealisation the session criticises; a student may ask "so where is the friction?", and the honest answer is "in the part the map does not have", which is the point. The Meshcat version has been started but not yet driven through a full grasp-and-release cycle by hand.

- **The `.g` scene is unverified on RAI** (no Linux runtime here); the syntax was checked against a model file and the tutorial, not by loading it. First thing to run on R 3's dry run.
- **Density of 2.2 (revised 2026-09-23).** Keep the frame-conversion direction explicit. Rotation encodings remain recognition-level material; use the existing rotations module instead of expanding into quaternion algebra. The encodings and the scene file are two slides again (29 and 30, 2026-09-28).
- **Notation additions for `notation.md` §2.1:** `R_x(θ), R_y(θ), R_z(θ)` elementary rotations; `[û]_×` skew matrix; `r = (w, x, y, z)` unit quaternion, `w` first (added on 2026-09-04). The rotation vector is written `θ û`, not `ω`, because `ω` is angular velocity. **Collision recorded in §3:** `θ` is the rotation angle here and the learned-parameter symbol in the learning block; they never share a slide, but a student reading both sets of notes side by side will notice. `ψ, ϑ_p, ϕ` for Euler angles are confined to §3.4 and §4.7.
- **The two-dimensional module cannot show that rotations do not commute** (planar rotations do). The order lesson leans on the numeric example in §4.5 and on the Drake notebook planned for session 2.4.
- **Videos.** No video is used today; deck 02 p.7 and p.21–23 were viewer recordings of the gripper "plan sketch" and are not needed for these two sessions.

Slide plan for this day (revised 2026-09-28; the presenter notes sum to 103 minutes)

The split of the encodings slide on 2026-09-28 added three minutes to session 2.2. Under the ruling of 2026-09-27, minutes are information and do not cut content; the block is 100 minutes.

Two 50-minute teaching sessions; the midpoint marks their boundary. Backup slides have no allocated teaching time. Presenter notes in `slides.md` contain the current teaching script; `essentials.md` supplies the revised conceptual explanations.

#	Slide	Minutes
1	Deck 02 · What learning replaces, and the notation for frames	1
2	Recap: the five boxes, and the map each one reads	3
3	The words on the map, read off one mug	3
4	What the classical stack assumes to know	2

#	Slide	Minutes
5	When the map is good enough: the robot by itself	3
6	Challenge 1a: interaction — locomotion	2
7	Challenge 1b: interaction — manipulation	4
8	Computed: how much the mug moves per joint	5
9	Challenge 2: state estimation	4
10	Which problems are easy, which are hard	3
11	What learning supplies, and what the stack keeps	4
12	Where these learned components appear in the course	3
13	What a learned component silently assumes	4
14	Three questions for every learned component, from now on	4
15	Summary of 2.1	5
16	Break	0
17	Why frames	3
18	A frame is an origin plus right-handed axes	3
19	Monogram notation: three slots, three facts	4
20	Reading exercise	3
21	The rotation matrix, derived in two dimensions	5
22	Rotation columns tell us which way to convert	3

#	Slide	Minutes
23	The three elementary rotations	3
24	Changing the reference frame: the one equation of today	4
25	Worked example (a): a mug on a tray on a table	4
26	Worked example (b), (c): same arrow, and the inverse problem	4
27	Live demo: the frames module	4
28	Composition: match frames, then check the order	3
29	One orientation, several encodings	4
30	A scene file lists each frame and its parent	3
31	Failure modes of the week	2
32	The next deck, and A1	1